

金融數據分析網頁工具開發

課前準備

金融數據分析網頁工具開發：課前準備指南

➤ Windows 版本

Visual Studio Code：

<https://code.visualstudio.com/download>

1. 下載 Windows 版本
2. 安裝 Visual Studio Code
3. 開啟 Visual Studio Code
4. 點選左邊的 Extensions
5. 安裝 Chinese (Traditional) Language Pack 中文套件
6. 安裝 Python 套件
7. 安裝 Remote - SSH 套件
8. 安裝 WSL 套件

WSL 虛擬機安裝：

<https://learn.microsoft.com/zh-tw/windows/wsl/install>

1. [以系統管理員身分執行] 命令提示字元
2. 輸入 `wsl --install`
3. 安裝完成後重開機 (重開機後會出現類似命令提示字元的視窗)
4. 建立使用者名稱 (全英文小寫) 和密碼

虛擬機指令視窗 (往後指令接下於此)：



1. 於 windows 搜尋 Ubuntu
2. 點擊開啟

套件安裝：

1. 指令 `sudo apt update && sudo apt upgrade`
2. 遇到 `[sudo] password for "使用者名稱"`：時輸入前面設定的密碼
3. 遇到 `Do you want to continue? [Y/n]` 時輸入 `y`
4. 指令 `sudo apt install python3-pip`

Windows 版本

- WSL 虛擬機安裝：

<https://learn.microsoft.com/zh-tw/windows/wsl/install>

1. [以系統管理員身分執行] 命令提示字元
2. 輸入 `wsl --install`
3. 安裝完成後重開機 (重開機後會出現類似命令提示字元的視窗)
4. 建立使用者名稱 (全英文小寫) 和密碼



Ubuntu

Windows 版本

- 套件安裝：

1. 指令 `sudo apt update && sudo apt upgrade`
2. 遇到 `[sudo] password for “使用者名稱”` :時輸入前面設定的密碼
3. 遇到 `Do you want to continue? [Y/n]` 時輸入 `y`
4. 指令 `sudo apt install python3-pip`



Ubuntu

MacBook 版本

- 安裝 Homebrew :

```
/bin/bash -c “$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)”
```

- 安裝 Python :

```
brew install python
```

- 查看 Python版本 :

```
python3 --version
```

軟體安裝

- Visual Studio 安裝

<https://code.visualstudio.com/download>

- DB Browser安裝（WSL）

1. 指令 `sudo apt update`
2. 指令 `sudo apt install sqlitebrowser`
3. 指令 `sqlitebrowser` 可開啟 DB Browser

- DB Browser安裝（MacBook）

<https://sqlitebrowser.org/dl/>

Linux 系統

- WSL 虛擬機即是 Windows 內建的簡易版 Linux。
- macOS 與 Linux 差異不大，所以 MacBook 使用者不必另外安裝虛擬機。
- Linux 常用指令：
 1. `ls`：列出檔案內資料
 2. `ls -a`：列出檔案內所有資料(包含隱藏檔案)
 3. `cd`：進入/退出資料夾
 4. `cd directory`：進入名為director的資料夾
 5. `cd ..`：退回上一層
 6. `cd ~` 或 `cd`：退回至家目錄

虛擬環境

- 建立虛擬環境是為了在開發過程中能有一個乾淨的工作環境，方便各套件的版本管理以及避免套件之間衝突。

- 安裝虛擬環境套件（MacBook免裝）：

```
sudo apt install python3-venv
```


虛擬環境

- 建立虛擬環境：

```
python3 -m {VENV NAME} venv
```

```
eg. python3 -m myvenv venv
```

- 進入虛擬環境：

```
. {VENV NAME}/bin/activate
```

```
eg. . myvenv/bin/activate
```

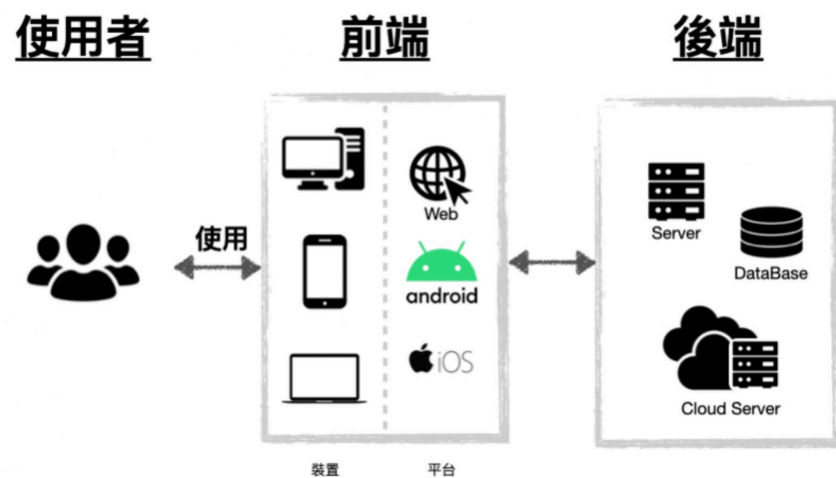
- 離開虛擬環境：

```
deactivate
```

建立Django專案

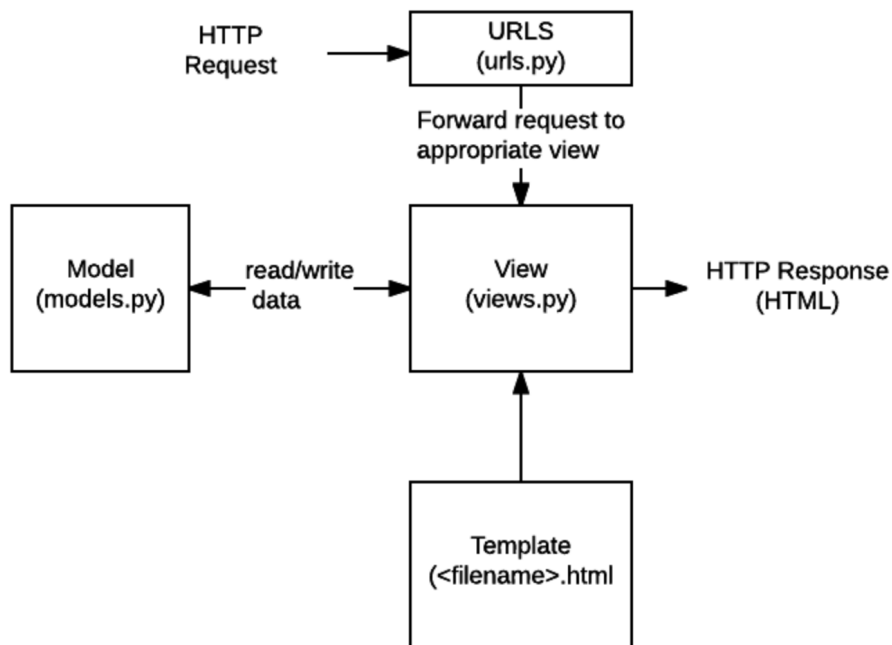
網站基本架構

- 前端：一般使用者透過瀏覽器或 APP 所看到的各種資料與美麗頁面。
- 後端：負責連接伺服器 (Server) 與資料庫 (Database) 的地方，有時候還需要整合前端功能，進行後台的各種資料運算、分析，再將處理好的資訊送去給前端渲染 (render)，讓一般使用者能看到結果。



Django 簡介

- Django 使用了 MTV 架構（Model、Template、View）
- Model：資料模組
- Template：視圖模組
- View：控制模組



建立 Django 專案

- 安裝 Django :

```
pip install Django
```

- 開始新專案 :

```
django-admin startproject {PROJECT NAME}
```

```
eg. django-admin startproject NCKU_Modular_Course
```

建立 Django 專案

- 專案建立成功後，進入專案會看到如右圖的結構。

```
.  
├── manage.py  
└── NCKU_Modular_Course  
    ├── asgi.py  
    ├── __init__.py  
    ├── settings.py  
    ├── urls.py  
    └── wsgi.py
```

- `manage.py`：此 Django 專案內的主要管理程式碼，當我們今天要新增 app 或是啟動 server 都要透過它。
- `Lab_Training` 資料夾：與專案名稱相同，負責放一些全域設定。
- `__init__.py`：定義 python 模組。
- `settings.py`：整個專案的全域設定，包含套件管理、app 註冊、templates 連結、time zone 等等都會在這裡作設定。
- `urls.py`：網址設定檔，設定每個 URL 要對應到的函式。通常每新增一個網頁就需要編輯它。
- `wsgi.py` or `asgi.py`：讓網站上線所需要的介面設定檔，Apache Server 會去讀取此檔案。

建立 Django 專案

- 進入專案資料夾：

`cd {Your Project Name}`

- 啟動專案：

`python manage.py runserver`

- 成功啟動將顯示：

```
Watching for file changes with StatReloader
Performing system checks...

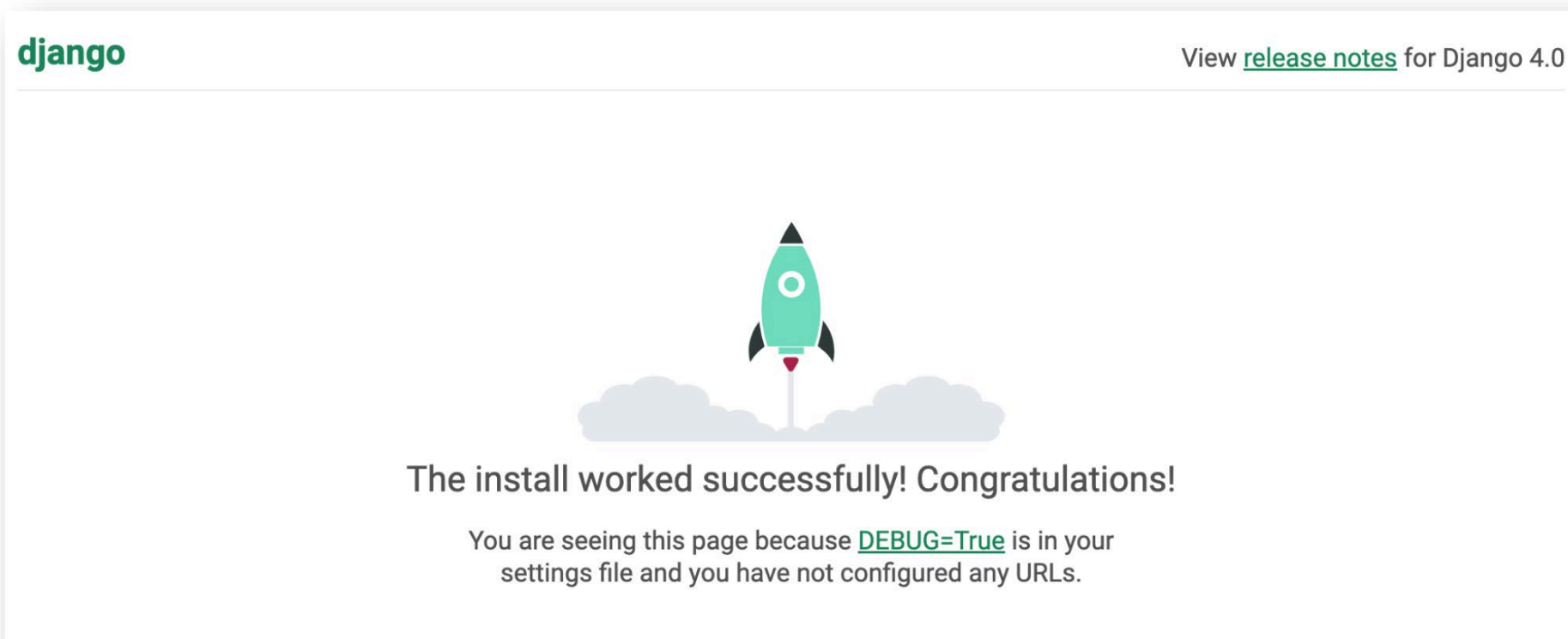
System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): ad
Run 'python manage.py migrate' to apply them.

Django version 4.0.4, using settings 'Lab_Training.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

建立 Django 專案

- 打開瀏覽器並輸入 <http://127.0.0.1:8000/> 或 <http://localhost:8000/>



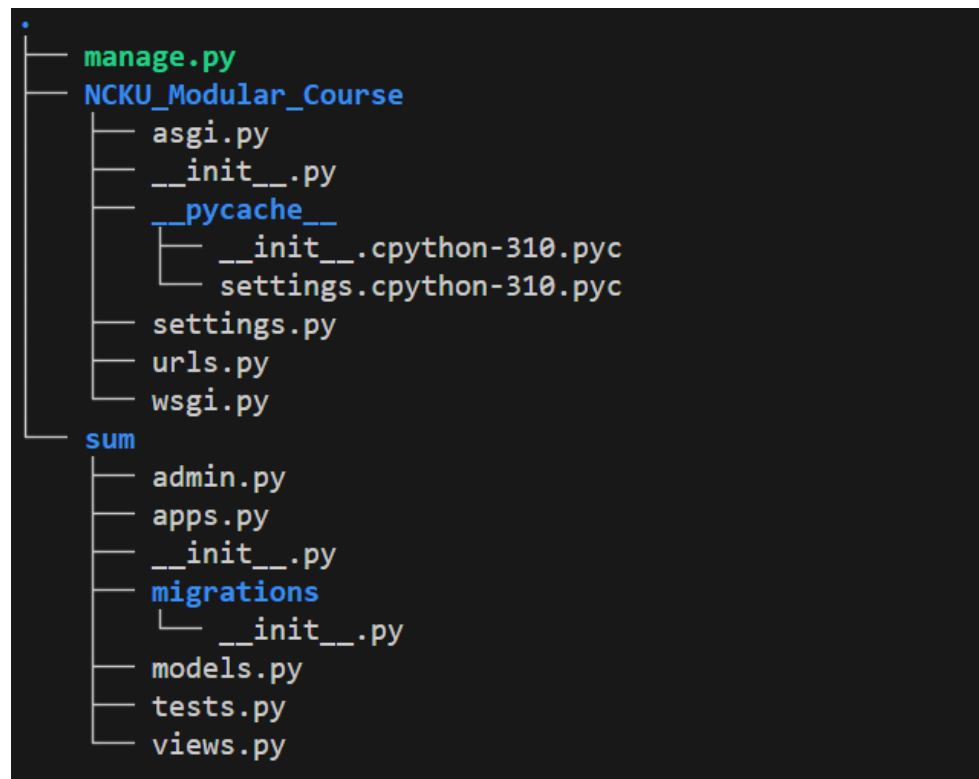
建立前後端溝通網站

建立網站

- 一個專案可以同時管理很多網站，可以將一個 app 理解成一個網站。
- 建立第一個叫 sum 的 app：

`python manage.py startapp sum`

- 成功後會產生一個叫 sum 的資料夾



建立網站

- 在專案中新增 sum 這個 app：
在 settings.py 修改

```
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'sum', # <= Add the sum app to the project
]
```

- 修改系統語言設定與時區：
在 settings.py 修改

```
# Internationalization
# https://docs.djangoproject.com/en/5.1/topics/i18n/

LANGUAGE_CODE = 'zh-Hant' # <= Change the language code to zh-Hant

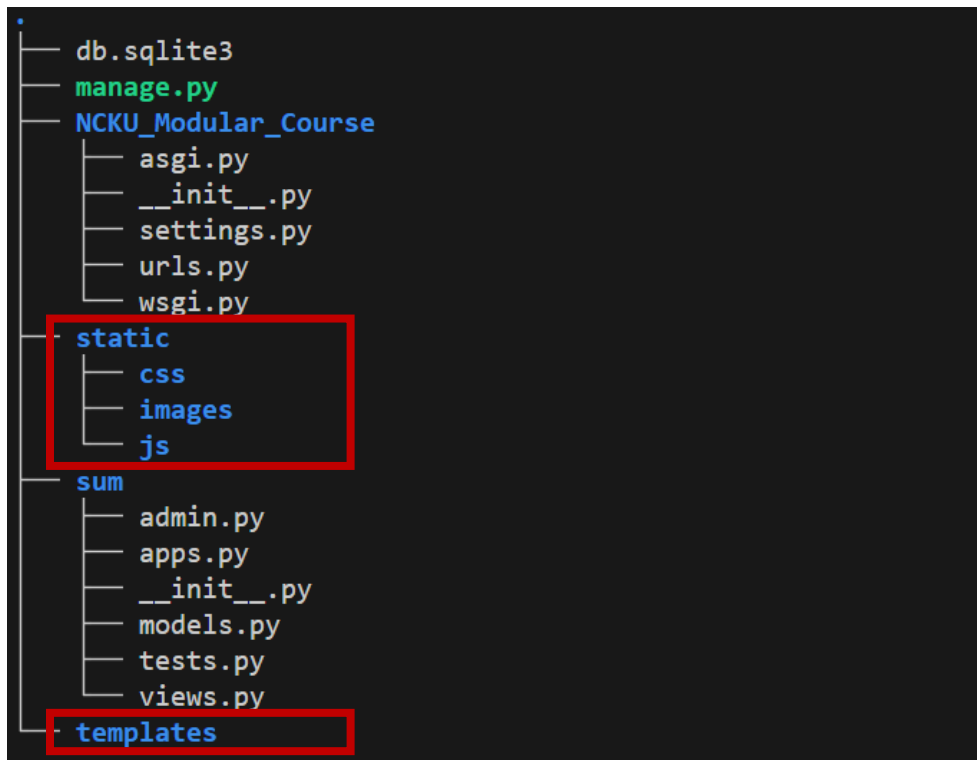
TIME_ZONE = 'Asia/Taipei' # <= Change the time zone to Asia/Taipei

USE_I18N = True

USE_TZ = True
```

建立網站

- 在專案下新增兩個資料夾 templates 和 static，static 下再新增 js、css、images 三個資料夾。



建立網站

- 設定 templates 資料夾路徑：
在 settings.py 修改

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [BASE_DIR / 'templates'], # <= Add the templates directory to the project  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    ],  
]
```

- 設定 static 資料夾路徑：
在 settings.py 修改

```
# Static files (CSS, JavaScript, Images)  
# https://docs.djangoproject.com/en/5.1/howto/static-files/  
  
STATIC_URL = 'static/'  
STATICFILES_DIRS = [BASE_DIR / "static"] # <= Add the static directory to the project
```

建立前端 html

- 在 templates 資料夾內新增 sum.html
- {% load static %} 導入 static 資料夾
- 載入 jQuery
- 載入 sum.css
- 載入 sum.js
- 設計自己的網頁頁面

```
{% load static %}

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sum</title>

    <!-- 載入 jQuery -->
    <script src="https://code.jquery.com/jquery-3.6.4.min.js"></script>
    <!-- 載入 sum.css -->
    <link rel="stylesheet" href="{% static 'css/sum.css' %}">

</head>
<body>
    <h1>加法計算器</h1>
    <form onsubmit="return false;">
        <input type="number" id="num1" placeholder="輸入第一個數字">
        <span>+</span>
        <input type="number" id="num2" placeholder="輸入第二個數字">
        <button id="btn_calculateSum">計算</button>
    </form>
    <div class="result" id="result"></div>

    <!-- 載入 sum.js -->
    <script src="{% static 'js/sum.js' %}"></script>

</body>
</html>
```

建立後端 views.py

- 在 sum/views.py 中撰寫程式
- def sum(request):
讓 sum.html 顯示
- def ajax_sum(request):
用來計算 sum

```
from django.shortcuts import render
from django.http import JsonResponse

# Create your views here.
def sum(request):
    return render(request, 'sum.html', locals())

def ajax_sum(request):
    a = int(request.GET['num1'])
    b = int(request.GET['num2'])

    response = {
        'sum': a+b
    }
    return JsonResponse(response)
```

設定網址 urls.py

- 在 NCKU_Modular_Course/urls.py 中撰寫

```
from django.contrib import admin
from django.urls import path
from sum import views # <= Add the sum/views.py module to the project

urlpatterns = [
    path('admin/', admin.site.urls),
    path('sum/', views.sum), # <= 這個網址用作顯示 sum.html
    path('ajax_sum/', views.ajax_sum), # <= 這個網址用作處理 ajax 請求
]
```


建立前端 js

- 在 static/js 資料夾內新增 sum.js
- 設定使用者與網頁互動的方式
- ajax：讓前後端互通的主要程式碼，用來將前端使用者輸入的數字傳給後端 views.py 中的 def ajax_sum 做加法運算，並將結果回傳給前端

```
$(document).ready(function(){

    $('#btn_calculateSum').click(function() {
        // 獲取使用者輸入的數字
        const num1 = parseFloat(document.getElementById('num1').value);
        const num2 = parseFloat(document.getElementById('num2').value);

        // 驗證輸入
        if (isNaN(num1) || isNaN(num2)) {
            document.getElementById('result').innerText = '請輸入有效的數字！';
            return;
        }

        const formData = {};
        formData['num1'] = num1;
        formData['num2'] = num2;

        $.ajax({
            url: '/ajax_sum/',
            method: 'GET',
            data: formData,
            success: function(response){
                sum = response['sum'];
                // 顯示結果
                document.getElementById('result').innerText = `結果：${sum}`;
            },
            error: function(xhr, status, error) {
                console.error('AJAX Error:', status, error);
            }
        });
    });

});
```

建立前端 CSS

- 在 static/css 資料夾內新增 sum.css
- 美化前端網頁

```
body {  
    font-family: Arial, sans-serif;  
    margin: 50px;  
    text-align: center;  
}  
input {  
    margin: 10px;  
    padding: 5px;  
    font-size: 16px;  
}  
button {  
    padding: 5px 10px;  
    font-size: 16px;  
}  
.result {  
    margin-top: 20px;  
    font-size: 18px;  
    color: green;  
}
```

完整結構

```
.
├── db.sqlite3
├── manage.py
├── NCKU_Modular_Course
│   ├── asgi.py
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── static
│   ├── css
│   │   └── sum.css
│   ├── images
│   └── js
│       └── sum.js
├── sum
│   ├── admin.py
│   ├── apps.py
│   ├── __init__.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
├── templates
│   └── sum.html
```

檢視自製網頁

- 啟動專案：

`python manage.py runserver`

- 打開瀏覽器並輸入 <http://127.0.0.1:8000/sum/>

